

# ngrok

## The Complete Guide — Zero to Hero

---

Install ngrok, authenticate it, expose local HTTP and TCP services to the internet, set up a persistent config file, use custom static domains, and follow good security practices.

# Table of Contents

1. What is ngrok?
2. Installing ngrok (Windows / macOS / Linux)
3. Creating an Account & Getting Your Authtoken
4. Connecting Your Authtoken
5. Basic Usage — Exposing an HTTP Server
6. Exposing TCP Services (SSH, game servers, custom sockets)
7. The ngrok.yml Configuration File
8. Static (Reserved) Domains and Addresses
9. Useful Commands Cheat Sheet
10. Security Best Practices
11. Troubleshooting

# 1. What is ngrok?

ngrok is a tool that creates a secure tunnel from a public URL (or public TCP address) on the internet, straight to a service running on your local machine — even if that machine is behind NAT, a firewall, or has no public IP at all. It is commonly used for:

- Testing webhooks (Stripe, GitHub, Telegram bots, etc.) against a local dev server
- Sharing a local web app with a client or teammate temporarily
- Exposing a local API for mobile app testing
- Forwarding TCP services such as SSH, game servers, or custom sockets
- Quick remote access to a machine without configuring router port-forwarding

How it works, in short: ngrok runs an agent on your machine. That agent opens an outbound connection to ngrok's cloud edge servers. Traffic that hits your public ngrok URL travels through ngrok's servers and back down that same outbound connection to your local port. Because the connection is outbound-initiated, you don't need to open any ports on your router or firewall.

## 2. Installing ngrok

### Windows

- 1 Go to <https://ngrok.com/download> and download the Windows zip.
- 2 Extract it — you'll get a single file: `ngrok.exe`.
- 3 Move it somewhere convenient, e.g. `C:\ngrok\ngrok.exe`, and optionally add that folder to your `PATH` so you can run ngrok from any terminal.

Or, with a package manager:

```
choco install ngrok
# or
winget install ngrok.ngrok
```

### macOS

```
brew install ngrok/ngrok/ngrok
```

### Linux (Debian/Ubuntu)

```
curl -sSL https://ngrok-agent.s3.amazonaws.com/ngrok.asc \
| sudo tee /etc/apt/trusted.gpg.d/ngrok.asc >/dev/null
echo "deb https://ngrok-agent.s3.amazonaws.com buster main" \
| sudo tee /etc/apt/sources.list.d/ngrok.list
sudo apt update && sudo apt install ngrok
```

## Linux (generic binary — works on any distro)

```
curl -O https://bin.equinox.io/c/bNyj1mQVY4c/ngrok-v3-stable-linux-amd64.tgz
tar -xvzf ngrok-v3-stable-linux-amd64.tgz
sudo mv ngrok /usr/local/bin/
```

Verify the install by running:

```
ngrok version
```

## 3. Creating an Account & Getting Your Authtoken

- 1 Go to <https://dashboard.ngrok.com/signup> and create a free account (email, Google, or GitHub sign-up all work).
- 2 After logging in, open the left sidebar and click "**Your Authtoken**" (under Getting Started).
- 3 Copy the token shown there — it's a long string unique to your account. Treat it like a password; anyone with it can open tunnels using your account.

The free plan is enough for almost everything in this guide: HTTP/TCP tunnels, one static domain, and a handful of concurrent tunnels.

## 4. Connecting Your Authtoken

Before ngrok can open tunnels, it needs to know who you are. Run this once (it's saved locally so you only do it a single time per machine):

```
ngrok config add-authtoken YOUR_TOKEN_HERE
```

This writes the token into ngrok's config file, typically located at:

- Linux/macOS: `~/.config/ngrok/ngrok.yml`
- Windows: `%HOMEPATH%\AppData\Local\ngrok\ngrok.yml`

## 5. Basic Usage — Exposing an HTTP Server

Suppose you have a local web app running on port 3000 (React, Flask, Express, whatever). To expose it to the public internet:

```
ngrok http 3000
```

ngrok will print a terminal dashboard similar to this:

```
Session Status online
Account you@example.com (Plan: Free)
Version 3.x.x
```

```
Region United States (us)
Forwarding https://a1b2-203-0-113-5.ngrok-free.app -> http://localhost:3000
```

```
Connections ttl opn rt1 rt5 p50 p90
0 0 0.00 0.00 0.00 0.00
```

Anyone who opens that `https://...ngrok-free.app` URL reaches your local port 3000, wherever your machine physically is. Press `Ctrl+C` to stop the tunnel — the URL stops working immediately.

### Other common HTTP variants:

```
ngrok http 3000 --domain=myapp.ngrok-free.app # use a reserved domain (see section 8)
ngrok http https://localhost:8443 # forward to a local HTTPS server
ngrok http 3000 --basic-auth="user:password" # add HTTP basic auth in front of the tunnel
```

## 6. Exposing TCP Services (SSH, game servers, custom sockets)

HTTP tunnels only work for web traffic. For anything else — SSH, a custom Python socket server (like the client/server scripts from earlier), Minecraft servers, databases, etc. — use a TCP tunnel:

```
ngrok tcp 22 # e.g. expose local SSH
ngrok tcp 1080 # expose a custom TCP service on port 1080
```

The output looks like this:

```
Forwarding tcp://4.tcp.ngrok.io:14837 -> localhost:1080
```

That means anyone (or any client script) connecting to `4.tcp.ngrok.io` on port 14837 is transparently forwarded to port 1080 on your machine. In a client/server script like the one you set up earlier, you'd set:

```
RHOST = "4.tcp.ngrok.io"
RPORT = 14837
```

Note that on the free plan, the TCP address and port are randomly assigned each time you start the tunnel (unless you reserve a static TCP address — see section 8).

## 7. The `ngrok.yml` Configuration File

Instead of typing flags every time, you can define named tunnels in a config file and launch several at once. Find your config path with:

```
ngrok config check
```

Example config file:

```
version: "2"
authtoken: YOUR_TOKEN_HERE
tunnels:
webapp:
proto: http
addr: 3000
ssh:
proto: tcp
addr: 22
mysocket:
proto: tcp
addr: 1080
```

Start every tunnel defined in the file at once:

```
ngrok start --all
```

Or start just one named tunnel:

```
ngrok start webapp
```

## 8. Static (Reserved) Domains and Addresses

By default, every time you start ngrok you get a brand-new random URL/address. If you want a fixed one that never changes:

### Static HTTP domain (free plan includes one):

- 1 In the dashboard, go to **Domains** and click **New Domain** to reserve a free `xxxx.ngrok-free.app` domain (or connect your own custom domain on a paid plan).
- 2 Use it when starting the tunnel:

```
ngrok http 3000 --domain=your-name.ngrok-free.app
```

### Static TCP address (paid plans):

```
ngrok tcp 1080 --remote-addr=1.tcp.ngrok.io:12345
```

## 9. Useful Commands Cheat Sheet

| Command                      | What it does                      |
|------------------------------|-----------------------------------|
| <code>ngrok http 3000</code> | Expose local port 3000 over HTTPS |

|                                                 |                                                      |
|-------------------------------------------------|------------------------------------------------------|
| <code>ngrok tcp 1080</code>                     | Expose local port 1080 as a raw TCP tunnel           |
| <code>ngrok config add-authtoken TOKEN</code>   | Save your account authtoken locally                  |
| <code>ngrok config check</code>                 | Show config file path and validate it                |
| <code>ngrok start --all</code>                  | Start every tunnel defined in <code>ngrok.yml</code> |
| <code>ngrok start NAME</code>                   | Start one named tunnel from <code>ngrok.yml</code>   |
| <code>ngrok http 3000 --basic-auth="u:p"</code> | Password-protect an HTTP tunnel                      |
| <code>ngrok http 3000 --domain=X</code>         | Use a reserved static domain                         |
| <code>ngrok diagnose</code>                     | Run connectivity diagnostics                         |
| <code>ngrok version</code>                      | Show installed version                               |

## 10. Security Best Practices

- Never share your authtoken publicly (e.g. in a GitHub repo or screenshot) — it grants control of your ngrok account.
- Add `--basic-auth` to HTTP tunnels that expose anything sensitive, since the public URL is guessable/scannable by anyone.
- Only forward the exact port you need — don't tunnel an entire machine's network stack.
- Turn off the tunnel (`Ctrl+C`) when you're done testing; leaving it running keeps that service exposed to the internet.
- For anything long-term or production-facing, prefer a proper reverse proxy with TLS and firewall rules instead of leaving a personal ngrok tunnel open indefinitely.

A public tunnel — HTTP or TCP — makes whatever you forward reachable by anyone on the internet, not just people you send the link to. Don't expose services with default or weak credentials.

## 11. Troubleshooting

| Problem                             | Likely fix                                                                                    |
|-------------------------------------|-----------------------------------------------------------------------------------------------|
| "authentication failed"             | Re-run <code>ngrok config add-authtoken</code> with a fresh token from the dashboard          |
| "tunnel not found" after restart    | Free-plan random URLs change every restart — reserve a static domain (section 8)              |
| Connection refused on the ngrok URL | Make sure your local service is actually running on the port you told ngrok to forward        |
| Slow tunnel / high latency          | Free tier uses shared regions; try <code>ngrok http 3000 --region=REGION</code> closer to you |

|                                          |                                                   |
|------------------------------------------|---------------------------------------------------|
| Windows Defender / firewall blocks ngrok | Allow ngrok.exe through Windows Defender Firewall |
|------------------------------------------|---------------------------------------------------|

---

Official docs: <https://ngrok.com/docs> — dashboard: <https://dashboard.ngrok.com>